OPERATING SYSTEMS

UE25CS242A

PES UNIVERSITY

CELEBRATING 50 YEARS

File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Driver Development, Flash Drives , File system implementation, Directory implementation

Pavan A C
Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

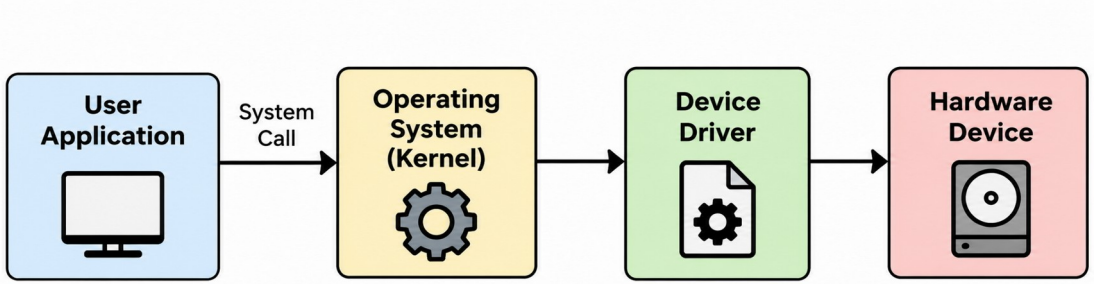
PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Device Driver

- A device driver is a specialized software component that enables the Operating System (OS) to communicate with and control a hardware device.
- It acts as a translator between the OS and the hardware.
- The OS does not need to understand the internal details of every hardware device.
- The driver provides a standard interface through which the OS can access the device.

Examples - Keyboard Driver, Mouse Driver, Printer Driver, Disk/Storage Driver, Network Card Driver, USB Driver



Types of Device Drivers & Architecture



1. Character Device Drivers

- Transfer data one character/byte at a time
- Used for devices such as keyboards, mice, and serial ports

2. Block Device Drivers

- Transfer data in fixed-size blocks
- Used for storage devices such as HDDs, SSDs, and USB drives

3. Network Device Drivers

- Enable communication between the OS and network hardware
- Used for Ethernet cards and Wi-Fi adapters

How Does a Device Driver Work?



Device Driver Workflow

1. **Application Request -** A user application requests an operation such as read, write, or print.
2. **System Call -** The application uses a system call to request the service from the OS.
3. **Kernel Processing -** The OS kernel receives the request and determines which device and driver are required.
4. **Device Driver -** The driver converts the OS request into device-specific commands.
5. **Hardware Operation -** The hardware performs the requested operation.
6. **Interrupt / Response**
   - The device may generate an interrupt when the operation is completed.
   - The driver informs the OS, which then returns the result to the application.

Example

Application → read() → Kernel → Disk Driver → SSD/HDD → Interrupt → Kernel → Application

Driver Development in Linux



- Driver development is the process of creating software that allows the OS to communicate with and control a specific hardware device.
- In Linux, many device drivers run as part of the kernel.
- Drivers can be built into the kernel or loaded dynamically as Kernel Modules.

Basic Linux Driver Development

1. **Device Identification -** Understand the hardware and its communication interface.
2. **Driver Implementation -** Write the driver using C and Linux Kernel APIs.
3. **Driver Registration -** Register the driver with the Linux kernel.



4. Device Communication - Implement operations such as open, read, write and close.

5. Testing & Debugging - Test communication between the driver and hardware.

Example

User Program → System Call → Linux Kernel → Device Driver → Hardware

Flash Drives & Flash Memory



What is Flash Storage?

- Flash memory is a non-volatile memory that retains data even when power is turned off.
- It is widely used in USB flash drives, SSDs, memory cards, and smartphones.
- Flash storage has no moving mechanical parts, making it faster and more resistant to physical shock than HDDs.

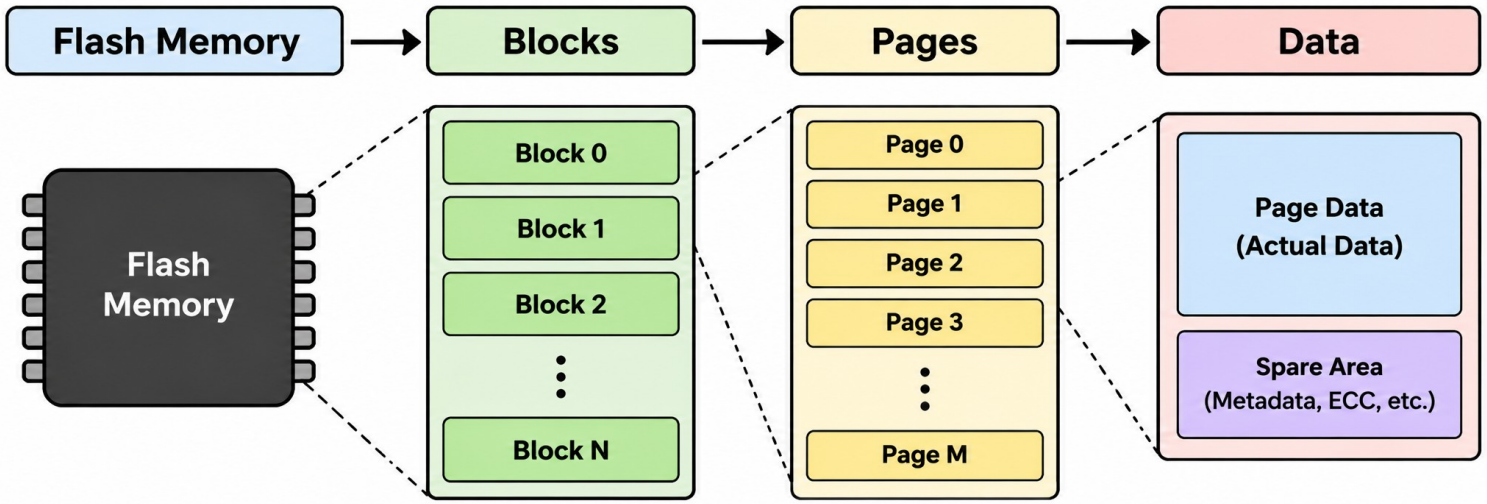
NAND Flash

- Most modern flash storage uses NAND flash memory.
- Data is organized into Pages and Blocks.
- Page: Smallest unit for read/write operations.
- Block: Collection of pages; erase operations occur at block level.

Flash Drives & Flash Memory

What is Flash Storage?

- Flash memory is a non-volatile memory that retains data even when power is turned off.
- It is widely used in USB flash drives, SSDs, memory cards, and smartphones.
- Flash storage has no moving mechanical parts, making it faster and more resistant to physical shock than HDDs.



NAND Flash

- Most modern flash storage uses NAND flash memory.
- Data is organized into Pages and Blocks.
- Page: Smallest unit for read/write operations.
- Block: Collection of pages; erase operations occur at block level.

Types of NAND Flash

Type	Bits / Cell	Key Feature
SLC	1 bit	Fast, reliable, high endurance
MLC	2 bits	Higher capacity, moderate endurance
TLC	3 bits	High capacity, widely used
QLC	4 bits	Very high capacity, lower endurance

NAND Flash



Flash Organization

- **Page:** Basic unit for read/write operations.
- **Block:** Collection of pages; erase operation is performed at block level.
- Multiple blocks form a **Flash Memory Chip**.

Important Concept

Read/Write → Page level

Erase → Block level

Example:

If a block contains 256 pages, data can be written to individual pages, but the entire block must be erased before its pages can be reused.

Flash Storage Management



Why is Flash Storage Management Required?

Flash memory has limitations:

- A flash cell has a limited number of write/erase cycles.
- Data cannot be overwritten directly like a magnetic disk.
- Erasing is performed at the block level, while writing is performed at the page level.

OPERATING SYSTEMS

Flash Storage Management

Key Management Techniques

1. Flash Translation Layer (FTL)

- Maps **logical addresses** from the OS to physical locations in flash memory.
- Hides the internal organization of flash from the OS.

2. Wear Leveling

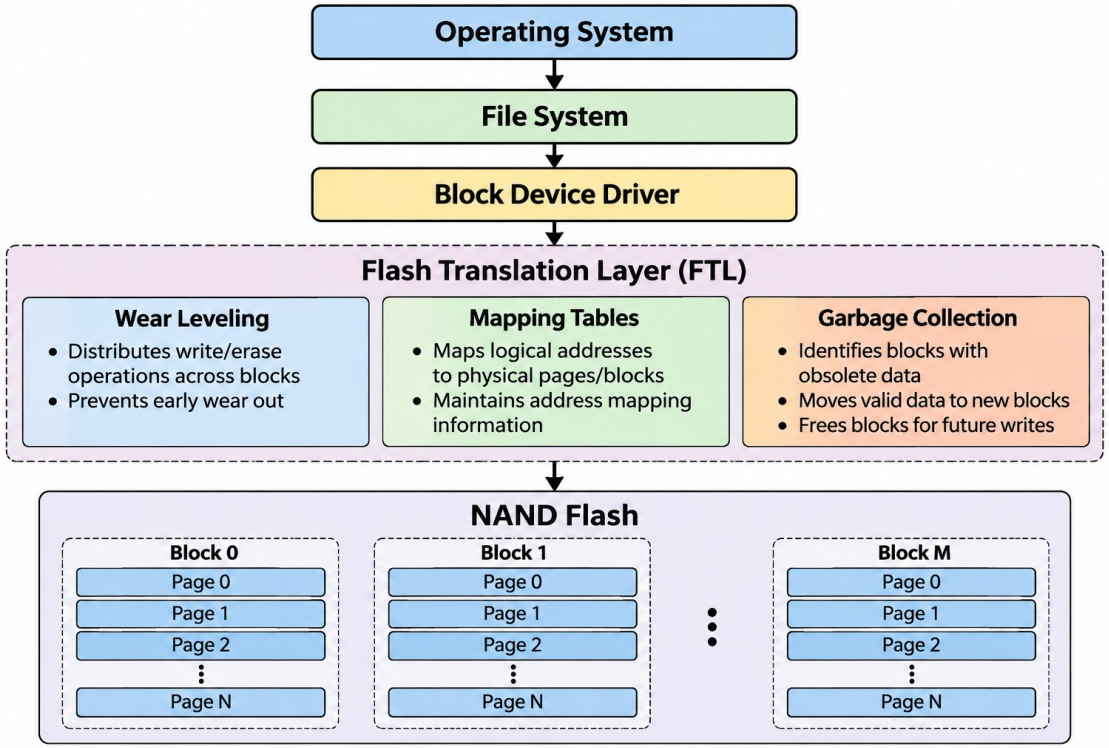
- Distributes write/erase operations across blocks.
- Prevents some blocks from wearing out earlier than others.

3. Garbage Collection

- Identifies blocks containing obsolete data.
- Moves valid data and frees blocks for future writes.

4. Bad Block Management

- Detects unusable flash blocks.
- Prevents the system from storing data in damaged blocks.



Flash Storage and the Operating System



How Does the OS Interact with Flash Storage?

- The OS views an SSD/flash drive as a block storage device.
- The file system manages files and directories using logical block addresses.
- The block device driver communicates with the storage device.
- The Flash Translation Layer (FTL) converts logical addresses into physical flash locations.

Important Operations

Read - OS requests data → Driver → FTL → Flash page → Data returned

Write - OS sends data → FTL selects a suitable page → Data is written

Erase - Flash memory erases entire blocks, not individual pages.

TRIM - The OS informs the SSD that certain blocks/pages are no longer in use and also, helps the SSD perform garbage collection more efficiently.

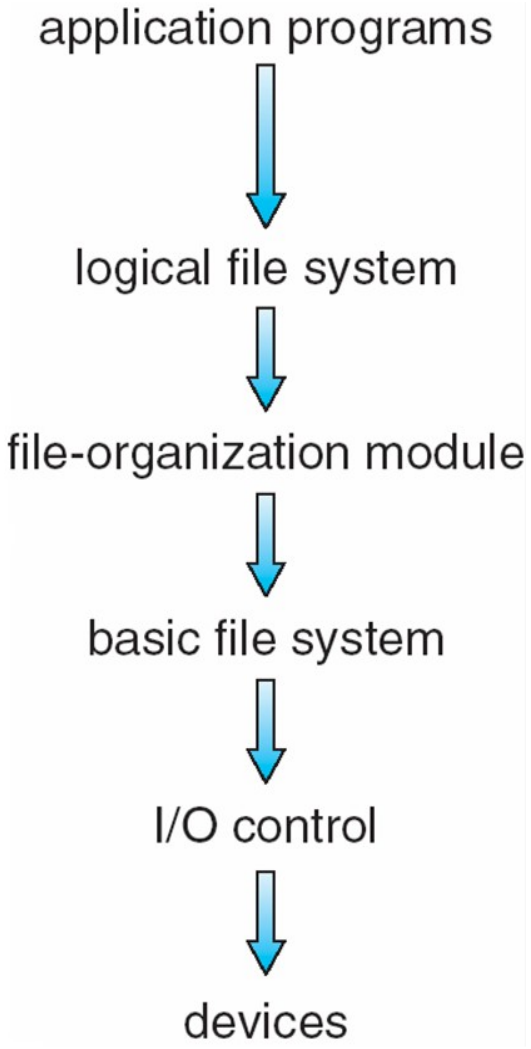
Flash Storage and the Operating System



Feature	HDD	SSD / Flash
Moving Parts	Yes	No
Access Time	Higher	Lower
Noise	Mechanical	Silent
Shock Resistance	Lower	Higher
Erase Requirement	No	Yes

OPERATING SYSTEMS

Layered File System



Manages metadata information, and the dir structure to provide the file-org module. Also responsible for protection

Translates logical to physical block addresses

Issues generic commands to the device driver, manages mem buffers

Device drivers and interrupt handlers

PES UNIVERSITY
CELEBRATING 50 YEARS

File-System Structure

- File structure
  - Logical storage unit
  - Collection of related information
- **File system** resides on secondary storage (disks)
  - Provided user interface to storage, mapping logical to physical
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- **File control block (FCB)** – storage structure consisting of information about a file including ownership, permissions, and location of the file contents
- **Device driver** controls the physical device
- File system organized into layers

PES UNIVERSITY
CELEBRATING 50 YEARS

File System Layers



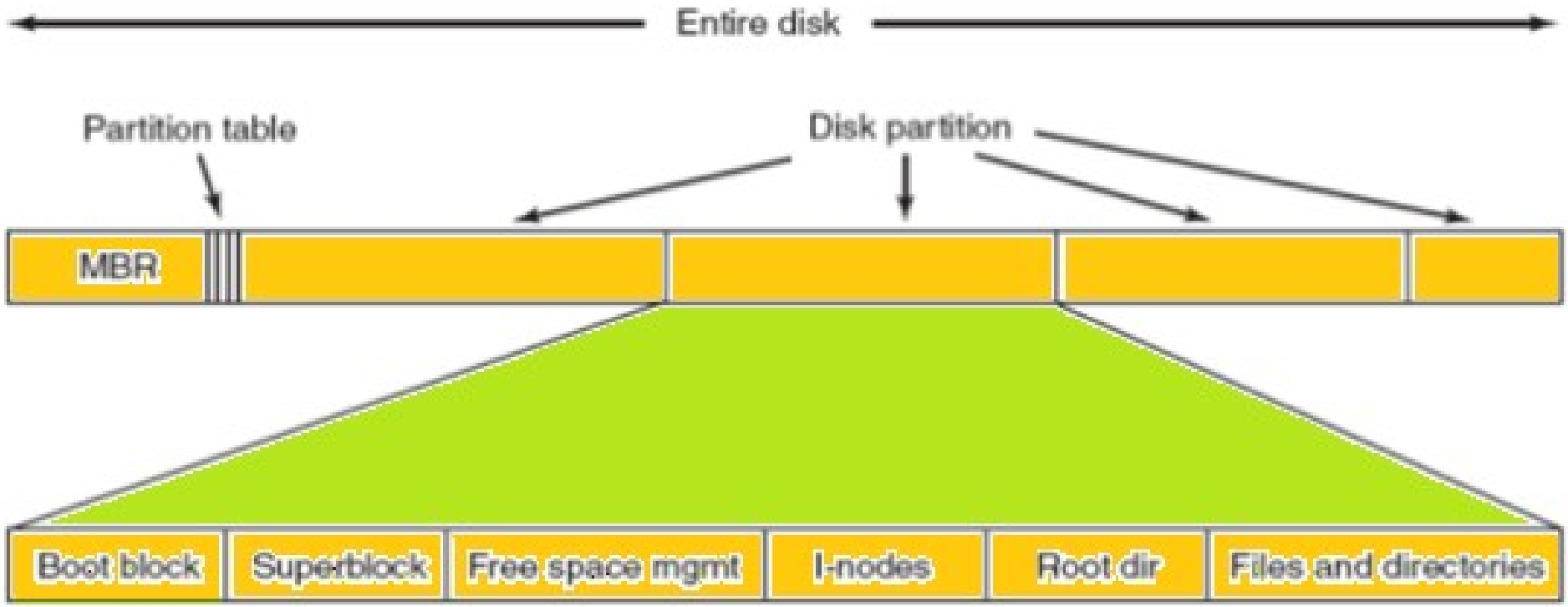
- **Device drivers** manage I/O devices at the I/O control layer
- **Basic file system** given command like “retrieve block 123” translates to device driver
- Also manages memory buffers and caches (allocation, freeing, replacement)
  - Buffers hold data in transit
  - Caches hold frequently used data

File System Layers (Cont.)



- **File organization module** understands files, logical address, and physical blocks
- Translates logical block # to physical block #
- Manages free space, disk allocation
- **Logical file system** manages metadata information
  - Translates file name into file number, file handle, location by maintaining file control blocks (**inodes** in UNIX)
  - Directory management
  - Protection

File-System Implementation



A possible file system layout.

File-System Implementation



- We have system calls at the API level, but how do we implement their functions?
  - On-disk and in-memory structures
- **Boot control block** contains info needed by system to boot OS from that volume
  - Needed if volume contains OS, usually first block of volume
- **Volume control block** (**superblock, master file table**) contains volume details
  - Total # of blocks, # of free blocks, block size, free block pointers or array
- Directory structure organizes the files
  - Names and inode numbers, master file table

OPERATING SYSTEMS

File-System Implementation (Cont.)

- Per-file **File Control Block** (**FCB**) contains many details about the file
  - inode number, permissions, size, dates
  - NFTS stores into in master file table using relational DB structures

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

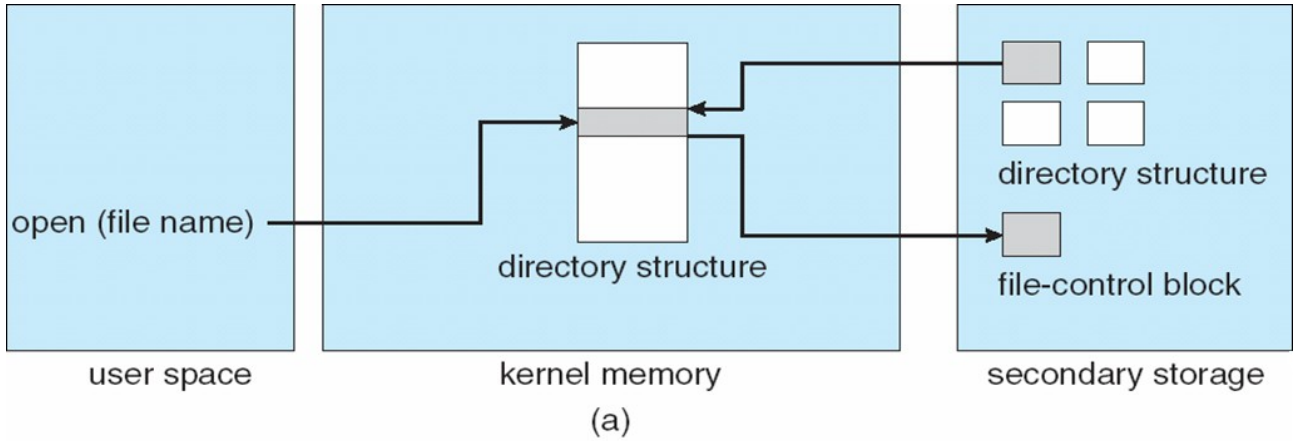
In-Memory File System Structures



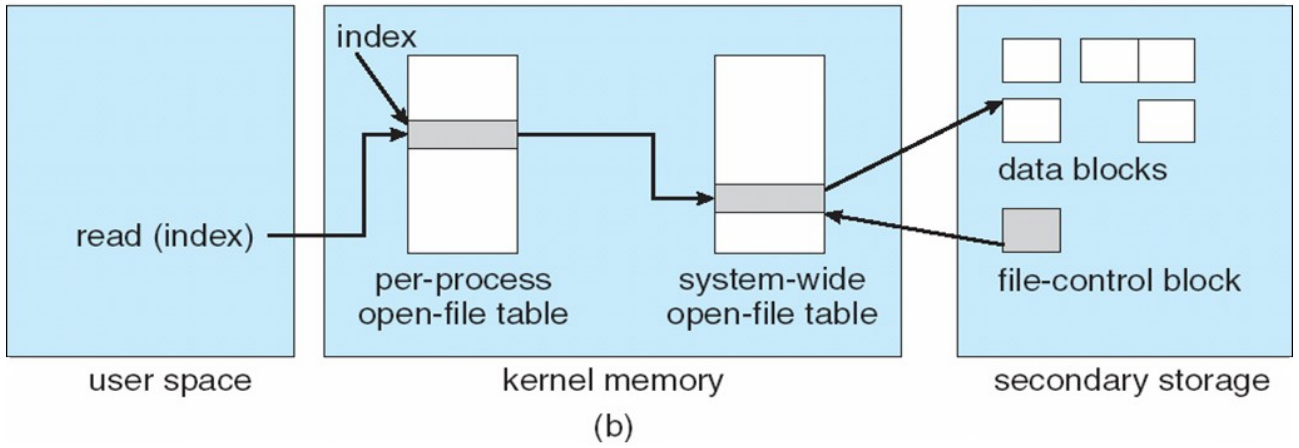
- Mount table storing file system mounts, mount points, file system types
- The following figure illustrates the necessary file system structures provided by the operating systems
- Figure (a) refers to opening a file
- Figure (b) refers to reading a file
- Plus buffers hold data blocks from secondary storage
- Open returns a file handle for subsequent use
- Data from read eventually copied to specified user process memory address

OPERATING SYSTEMS

In-Memory File System Structures



(a)



(b)

Virtual File Systems

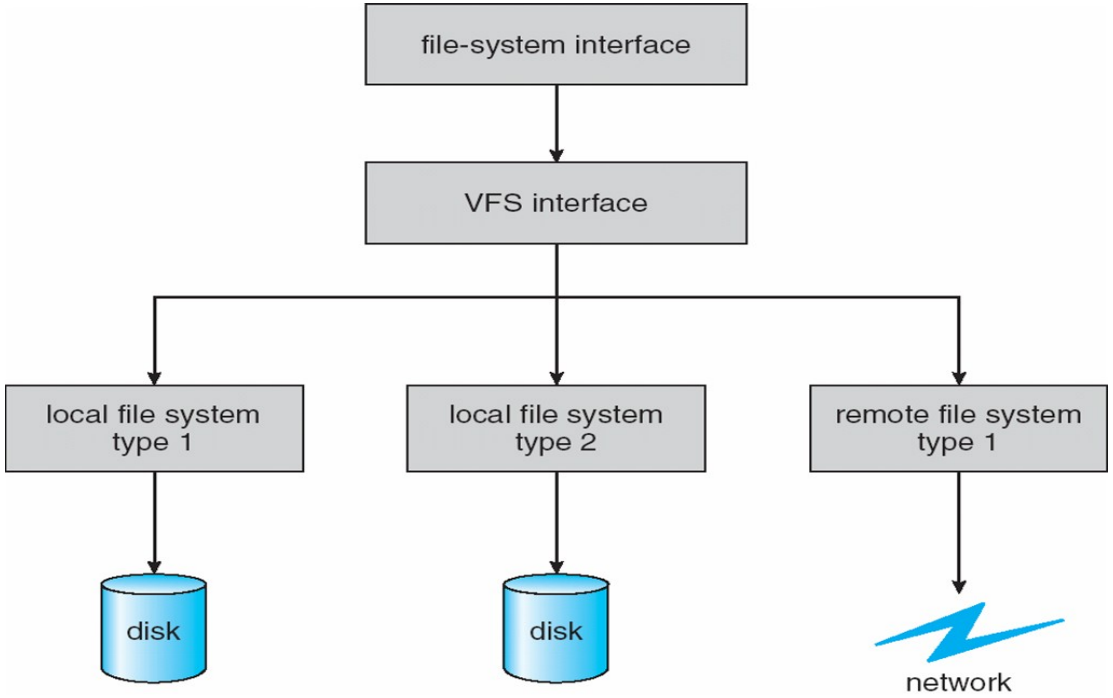


- **Virtual File Systems** (**VFS**) on Unix provide an object-oriented way of implementing file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems
  - Separates file-system generic operations from implementation details
  - Implementation can be one of many file systems types, or network file system
    - Implements **vnodes** which hold inodes or network file details
  - Then dispatches operation to appropriate file system implementation routines

OPERATING SYSTEMS

Virtual File Systems

- The API is to the VFS interface, rather than any specific type of file system (FS)
- VFS provides a single set of commands for the kernel and developers to access all types of FSs
- VFS software calls the specific device driver required to interface to various types of FSs
- Device driver interprets the standard set of FS commands to a specific type of FS on the partition or logical volume





THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu